

ITM 152: Object Oriented Programming in Java

BITM 2nd

Nature of the Course: Theory + Practical

Credits: 3

Lecture Hours: 48(Th) +16(Pr)

Course Description

This course provides a comprehensive introduction to Object Oriented Programming (OOP) using Java as the primary implementation language. Building upon the foundations of Structured Programming in C, students will transition to the object-oriented paradigm and develop proficiency in Java's type system, class hierarchy, inheritance, interfaces, exception handling, file handling, generics, and essential Java library classes.

The course bridges fundamental programming knowledge with modern software design principles, equipping students to design modular, reusable, and robust applications. Equal emphasis is placed on theoretical understanding and hands on laboratory work, preparing students for advanced software development courses and industry roles.

Course Objectives

By the end of this course, students will be able to

- Understand the history, features, and architecture of Java (JVM, JRE, JDK) and the object-oriented paradigm.
- Demonstrate knowledge of Java fundamentals including data types, variables, operators, and control statements.
- Define and use classes, objects, constructors, methods, and access control to implement encapsulation.
- Apply inheritance, method overriding, abstract classes, and interfaces to build extensible class hierarchies.
- Utilize arrays, strings, and essential Java library classes for solving computational problems.
- Handle exceptions effectively using Java's built-in and user-defined exception mechanisms.
- Implement file handling operations (text, binary, serialization) for data persistence.
- Apply generic programming concepts and use the Java Collections Framework.
- Write multi-threaded programs and manage concurrency issues.
- Develop complete Java applications using good software engineering practices

Course Details

Unit 1: Introduction to Java

5 LHs

The Creation of Java; Java's lineage and its influence on modern programming; Java's impact on the Internet; Java Buzzwords (Simple, Object-Oriented, Robust, Multithreaded, Architecture Neutral, Interpreted, High Performance, Distributed, Dynamic); Java Virtual Machine (JVM), Java Runtime Environment (JRE), and Java Development Kit (JDK) architecture and roles; The Bytecode; compilation model vs. interpretation model; Platform independence; Writing, Compiling, and Running Simple Java Programs using Command Line and IDE; Object-Oriented Programming Paradigm: Overview of OOP principles (Encapsulation, Inheritance, Polymorphism, Abstraction);

Using Command Line Arguments; Reading Input with Scanner; Writing Output with System.out

Learning Outcomes: Describe the history, features, and architecture of Java including JVM, JRE, and JDK (Knowledge); Write, compile, and execute simple Java programs using command line tools and IDE (Skills); Appreciate the significance of platform independence and OOP in Java application development (Competency).

Unit 2: Fundamental Programming Structures

6 LHs

Writing Comments; Primitive Data Types (byte, short, int, long, float, double, char, boolean); Variables and Constants; Type Conversion and Casting; Automatic Type Promotion Rules; Operators: Arithmetic, Bitwise, Relational, Logical (short-circuit), Assignment, Conditional (?:), sizeof equivalent; Operator Precedence and Associativity; Using Parentheses; Control Statements: if, if else, nested if else, else if ladder, switch; Iteration Statements: while, do while, for, enhanced for each loop, nested loops; Jump Statements: break, continue, return; Working with Big Numbers (BigInteger, BigDecimal overview); Declaration, Initialization, and Processing of One Dimensional and Multidimensional Array; Passing Arrays to Methods.

Learning Outcomes: Explain Java's data types, operators, and control structures with proper syntax (Knowledge); Write Java programs using decision making, iteration, and array processing (Skills); Apply fundamental programming structures to develop logical and efficient Java programs (Competency).

Unit 3: Classes and Objects

8 LHs

Object Oriented Principles: Encapsulation, Abstraction, Inheritance, Polymorphism; Defining Classes: fields, methods, and the general form of a class; Creating Objects and Accessing Class Members; Assigning Object Reference Variables; Methods: Adding Methods, Returning Values, Method Parameters and Return Types; Method Overloading; Constructors: Default and Parameterized; Constructor Overloading; the this Keyword; Variable Length Arguments; Access Control: public, private, protected, default; JavaBeans convention; static Fields and Methods; final Variables, Methods, and Classes; The Object Class; Nested and Inner Classes: static nested, inner, local, and anonymous classes; Garbage Collection and Finalization; Object Lifecycle; Creating and Using Packages; import statements; CLASSPATH.

Learning Outcomes: Describe the concepts of classes, objects, constructors, methods, and access control in Java (Knowledge); Implement well-structured Java classes with proper encapsulation, method overloading, and package organization (Skills); Apply OOP principles to design modular and reusable Java components (Competency).

Unit 4: Inheritance and Interfaces

7 LHs

Inheritance Basics: extends keyword, member access, IS-A relationship; Using super: calling superclass constructors and overridden methods; Method Overriding: rules, @Override annotation, covariant return types; Dynamic Method Dispatch: runtime polymorphism; why overridden methods matter; Abstract Classes: abstract methods and the template method concept; inheritance types; Using final with Inheritance(preventing overriding and subclassing); Interfaces: defining, implementing, and extending interfaces;

Default and Static Methods in Interfaces (Java 8+); multiple interface inheritance; Functional Interfaces and the `@FunctionalInterface` annotation; Sealed Classes overview; Liskov Substitution Principle.

Learning Outcomes: Explain inheritance, method overriding, abstract classes, and interfaces in Java (Knowledge); Implement class hierarchies using inheritance, polymorphism, and interfaces effectively (Skills); Apply inheritance and interface-based design to build extensible and loosely coupled Java applications (Competency).

Unit 5: Exception Handling

4 LHs

Exception Handling Fundamentals; Exception Hierarchy: Throwable, Error, Exception, RuntimeException; Checked vs. Unchecked Exceptions; understanding uncaught exceptions and stack traces; Using try, catch, and finally blocks; multiple catch clauses; nested try statements; Multi catch (Java 7+); try with resources for automatic resource management; throw and throws keywords; exception chaining (initCause, getCause); Java's Built in Exceptions overview; Creating Custom Exception Subclasses; best practices for exception design

Learning Outcomes: Describe the exception hierarchy and types of exceptions in Java (Knowledge); Implement robust exception handling using try catch finally, custom exceptions, and try with resources (Skills); Apply exception handling to build fault tolerant programs that handle errors gracefully (Competency).

Unit 6: Generics and Collections

5 LHs

Importance of Generic Programming; defining Generic Classes and Methods; type parameters; Bounded Wildcards (extends/super); type erasure and heap pollution; Restrictions and Limitations of Generics; Inheritance Rules with Generics; The Java Collections Framework: List, Set, Map, Queue, and Deque interfaces; Iterators and the Iterable interface; enhanced for loop with collections; Type Wrappers: Character, Boolean, numeric wrappers; Autoboxing and Unboxing; Autoboxing in expressions; autoboxing with Boolean and Character; preventing errors.

Learning Outcomes: Describe generic programming concepts, type parameters, wildcards, and the Collections Framework (Knowledge); Implement generic classes and methods (Skills); Apply generics and collections to write type-safe, reusable, and efficient data management code (Competency).

Unit 7: Multithreaded Programming

5 LHs

The Java Thread Model: threads vs. processes; benefits of multithreading; Thread Priorities; synchronization; messaging between threads; The Thread Class and the Runnable Interface; the main thread; Creating Threads: implementing Runnable; extending Thread class; Creating Multiple Threads; using `isAlive()` and `join()`; Thread Priorities and scheduling; Synchronization: synchronized methods and synchronized statements.

Learning Outcomes: Explain the Java thread model, thread lifecycle, synchronization, and enumeration concepts (Knowledge); Create and manage threads using Runnable and Thread; apply synchronized mechanisms to prevent race conditions (Skills); Apply multithreading techniques to develop concurrent, efficient, and thread safe Java applications (Competency).

Unit 8: Essential Java Classes

3 LHs

String Class: constructors, length, special operators (concatenation, conversion, toString()); Character extraction: charAt() and getChars(); String comparison methods; Modifying Strings: substring(), concat(), replace(), trim(); StringBuffer and StringBuilder: mutable strings, performance considerations; String Handling Library Functions overview; Math Class: commonly used mathematical methods; Primitive Type Wrappers: Character, Boolean, Integer, Double methods and conversions; Introduction to Lambda Expressions and Stream API (overview for modern Java awareness).

Learning Outcomes: Describe the functionality of essential Java classes including String, StringBuilder, Math, and wrapper classes (Knowledge); Use string manipulation methods, math functions, and wrapper class utilities to solve common programming problems (Skills); Apply essential Java library classes to write cleaner, more expressive, and efficient programs (Competency).

Unit 9: File Handling

5 LHs

Introduction to Files; File class: creating, deleting, listing files and directories; File Modes and I/O Exceptions; AutoCloseable, Closeable, and Flushable Interfaces; Byte Streams: InputStream, OutputStream, FileInputStream, FileOutputStream; PrintStream, DataOutputStream, DataInputStream; RandomAccessFile for random access; Character Streams: Reader, Writer; FileReader, FileWriter, PrintWriter, BufferedReader, BufferedWriter; The Console Class; reading from standard input and writing formatted output; NIO.2 API (java.nio.file): Path, Files, Paths modern file operations; Sequential and Random Access to Files; Serialization: Serializable interface, ObjectOutputStream, ObjectInputStream; I/O Classes and Interfaces review.

Learning Outcomes: Describe Java file I/O concepts including file types, stream hierarchies, serialization, and NIO.2 (Knowledge); Implement file handling operations opening, reading, writing, appending, closing for both text and binary files using byte and character streams (Skills); Apply file management and serialization techniques to develop programs with persistent data storage and retrieval capabilities (Competency).

Pedagogical Strategies

- Lectures with live coding demonstrations
- Hands-on laboratory sessions with guided and open-ended exercises
- Problem-based learning using real-world scenarios
- Pair programming and peer code review activities
- Multimedia presentations to visualize OOP and concurrency concepts
- Mini project with iterative design and feedback cycles
- Guest lectures from software industry professionals
- Continuous assessment and formative feedback

Mode of delivery

- Lecture sessions (Theory)
- Demonstration
- Laboratory work (Practical)
- Mini project

Internal assessment methods and types (40%)

Assessment Type	Weightage	Details
Class participation & attendance	10%	Contribution to discussions, engagement in class activities
Quizzes/short tests	15%	Periodic quizzes to assess comprehension
Practical/Project	20%	Lab sessions and mini project
Mid-term examination	25%	Written test
Pre-board examination	30%	Comprehensive written test covering all units

External assessment methods and types (60%)

Out of the total 60% allocated for final assessment, 40% will be assigned to the written/board examination to evaluate students' abilities in remembering, understanding, applying, analyzing, evaluating, and creating. The remaining 20% will be assigned to the final practical examination to assess hands-on programming skills and competency.

Mapping course: Learning outcomes and program learning

Course Learning Objective (CLO) Dimensions	Knowledge (K)	Skills (S)	Competence (C)
Total Learning	35%	40%	25%

Laboratory work

Students are required to complete hands-on programming exercises for every unit. The following practical tasks are prescribed:

- Unit 1 Lab: Write and execute basic Java programs; explore JDK tools (javac, java); use Scanner for input
- Unit 2 Lab: Programs using operators, type casting, control statements, and arrays
- Unit 3 Lab: Design and implement classes with constructors, method overloading, static members, and packages
- Unit 4 Lab: Build class hierarchies using inheritance; demonstrate polymorphism and interface implementation
- Unit 5 Lab: Programs with try-catch-finally, custom exceptions, and try-with-resources
- Unit 6 Lab: Implement generic classes/methods; use ArrayList; and sorting with Comparator
- Unit 7 Lab: Create multithreaded programs; demonstrate synchronization and deadlock prevention; use Enumerations
- Unit 8 Lab: String processing programs; use StringBuilder, Math class, and wrapper classes
- Unit 9 Lab: File I/O programs — text file read/write, binary file handling, random access, and object serialization; develop a record-keeping system with search and manipulation features

Suggested Readings

Horstmann, C. S. (2022). *Core Java: Volume I—fundamentals* (12th ed.). Pearson Education.

Schildt, H. (2022). *Java: The complete reference* (12th ed.). McGraw Hill.

Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.

Deitel, P., & Deitel, H. (2020). *Java: How to program* (11th ed.). Pearson.

Loy, M., Niemeyer, P., & Leuck, D. (2020). *Learning Java: An introduction to real-world programming with Java*. O'Reilly Media.

Eckel, B. (2006). *Thinking in Java* (4th ed.). Prentice Hall.

Oracle Corporation. (2023). *Java SE 21 documentation*.
<https://docs.oracle.com/en/java/javase/21/>

TRIBHUVAN UNIVERSITY
FACULTY OF MANAGEMENT
Office of the Dean
Model Question 2026

Full Marks: 60
Pass Marks: 30
Credit Hour: 3
Time: 3 Hrs

BITM / Second Semester / ITM152: Object Oriented Programming in Java

Candidates are required to answer all the questions in their own words as far as practicable. Figures in brackets indicate full marks.

Group "A"
Concept Based Brief Answer Questions

Answer the following questions: (Attempt All)

[5 × 2 = 10]

- 1) What is the purpose of this keyword in Java?
- 2) Which interfaces and classes are used to serialize and deserialize an object?
- 3) List the difference between StringBuffer, and StringBuilder in Java.
- 4) List the Key Aspects of Java's Impact on the Internet.
- 5) What is the difference between throw and throws in Java exception handling?

Group "B"
Problem Solving / Short Answer Questions

Answer any SIX Questions:

[6 × 5 = 30]

- 6) Describe the architecture of JVM, JRE, and JDK.
- 7) Write a Java program to demonstrate type conversion and casting between different data types.
- 8) Explain method overloading and variable length arguments (varargs) in Java.
- 9) Explain the concept of Sealed classes with example
- 10) Write a Java program to demonstrate the use of nested try blocks.
- 11) What are wrapper classes in Java? Describe the concepts of autoboxing and unboxing.
- 12) Write a Java program using StringBuffer to demonstrate string manipulation by performing operations such as appending, inserting, and deleting characters.

Group "C"

Comprehensive / Extended Problem / Case / Situation Analysis / Long Answer Questions

Answer any TWO Questions:

[2 × 10 = 20]

- 13) Write a Java program to demonstrate file handling by performing read and write operations on a file.
- 14) Define polymorphism in Java and explain its types with suitable examples.
- 15) Write a Java program that simulates a ticket booking system in which multiple threads attempt to book tickets from a limited number of available seats. The program should include a class named TicketCounter with a method for booking tickets, and the total number of tickets should be initialized as 10 tickets. Create multiple threads using the Runnable interface that try to book tickets at the same time. Apply proper synchronization to ensure that tickets are not overbooked. The program should display which thread successfully books tickets and the number of tickets remaining after each transaction. Use the Thread class to create and manage the threads.